

1 In order to generate C you need a k and m, however if we assume that $|\Sigma.K| < |\Sigma.M|$ then there must exist some m such that there is no k so that $\text{enc}(k, m') = c$. Even if the enc function is randomized the lack of 1 to 1 mapping still means that an attacker would have an increased probability of knowing the original m from the C.

Here with an example where $m=(a,b,c,d)$ $k=(1,2,3)$ and $c=(A,B,C,D)$ An attacker that obtained a ciphertext would know be able to eliminate some of the possible plaintext thus violating one time secrecy.

	1	2	3
a	A	B	C
b	B	C	D
c	C	D	A
d	D	A	B

2 Assuming that Σ satisfies OTS then if $\mathcal{L}_{\text{ots-L}}^{\Sigma}$ is interchangeable with $\mathcal{L}_{\text{ots-R}}^{\Sigma}$ then Σ^2 satisfies OTS as well.

Starting with

```

 $\mathcal{L}_{\text{ots-L}}^{\Sigma^2}$ 
EAVESDROP( $m_L, m_R$ )
 $k_1 \leftarrow \Sigma.K$ 
 $k_2 \leftarrow \Sigma.K$ 
 $c_1 \leftarrow \Sigma.\text{Enc}(k_1, m_L)$ 
 $c_2 \leftarrow \Sigma.\text{Enc}(k_2, m_L)$ 
return ( $c_1, c_2$ )

```

Because the same enc function is used and each key is generated independently using the same keygen function then functionally using the same subroutine CTXT to generate both c1 and c2 does not change the functionality of the program.

```

 $\mathcal{L}_{\text{ots-L}}^{\Sigma^2}$ 
EAVESDROP( $m_L, m_R$ )

```

C1 := CTXT(m_L)

C2 := CTXT(m_L)

Return($c1, c2$)



```

 $\mathcal{L}_{\text{ots-R}}^{\Sigma}$  CTXT( $m$ )

```

$k \leftarrow \Sigma.K$

$c \leftarrow \Sigma.\text{Enc}(k, m)$

return C

Now because I am assuming that Σ satisfies OTS then I can replace the right side to this

$$\frac{\mathcal{L}_{\text{ots-L}}^{\Sigma^2}}{\text{EAVESDROP}(m_L, m_R)}$$

C1 := CTXT(m_L)

C2 := CTXT(m_L)

Return(c1,c2)

$$\mathcal{L}_{\text{ots-rand}}^{\Sigma} \text{CTXT}(m)$$

c ← Σ . C

return C

Because m has no affect on C that means I can change what I am assigning it to. Changing L to R in this case changes nothing functionally.

$$\frac{\mathcal{L}_{\text{ots-L}}^{\Sigma^2}}{\text{EAVESDROP}(m_L, m_R)}$$

C1 := CTXT(m_R)

C2 := CTXT(m_R)

Return(c1,c2)

$$\mathcal{L}_{\text{ots-rand}}^{\Sigma} \text{CTXT}(m)$$

c ← Σ . C

return C

Now I can undo the right side

$$\frac{\mathcal{L}_{\text{ots-L}}^{\Sigma^2}}{\text{EAVESDROP}(m_L, m_R)}$$

C1 := CTXT(m_R)

C2 := CTXT(m_R)

Return(c1,c2)

$$\mathcal{L}_{\text{ots-rand}}^{\Sigma} \text{CTXT}(m)$$

k ← Σ . K

c ← Σ . Enc (k , m)

return C

Now unfactoring the subroutine and putting the lines back to inline would result in the desire outcome.

$$\frac{\mathcal{L}_{\text{ots-R}}^{\Sigma^2}}{\text{EAVESDROP}(m_L, m_R)}$$

$$k_1 \leftarrow \Sigma . \mathcal{K}$$

$$k_2 \leftarrow \Sigma . \mathcal{K}$$

$$c_1 \leftarrow \Sigma . \text{Enc}(k_1, m_R)$$

$$c_2 \leftarrow \Sigma . \text{Enc}(k_2, m_R)$$

$$\text{return } (c_1, c_2)$$